When IOTLB Hits Wait: FPGA Characterization of Multi-DMA Interference in a RISC-V IOMMU

Anonymous Authors

***Abstract*—Shared I/O translation can delay a device even when its own address translations hit in the cache. This paper characterizes a fixed open RISC-V input/output memory management unit on a field-programmable gate array (FPGA) with autonomous concurrent direct-memory-access traffic. A zero-miss co-run control separates ordinary shared-path loss from the additional loss observed when the neighbor's translation working set grows. Segmented observers locate waiting, and a bounded intervention on a named page-table-walk response traces waiting propagation to a neighboring request that subsequently hits. A repaired baseline provides 100 board trials across direct wiring and an added-delay diagnostic configuration. The latter is a sensitivity experiment, rather than a measurement of pure page-table-walk cost. Historical capacity and mitigation experiments expose distinct replacement effects and costs to the interfering stream; an invalidation-scope study is reported separately as simulation. Device-count scaling, workload-shape and page-size coverage, and request-latency distributions remain explicit gaps in this working draft.**

***Index Terms*—IOMMU, RISC-V, DMA, FPGA, interference, address translation**

I. Introduction

An input/output memory management unit (IOMMU) lets direct memory access (DMA) devices use translated addresses while enforcing access permissions [1]. Translation also introduces a shared service path. A device can reuse its own I/O translation lookaside buffer (IOTLB) entries and still wait behind another device's misses. Prior work has already established IOMMU interference, shared-translation bottlenecks, and mechanisms for hit-under-miss service [2], [3], [4]. The performance question addressed here is more specific: in a fixed, executable RISC-V implementation, where does the waiting occur, which neighboring operation can extend it, and what does protecting one stream cost the other?

Several closely related studies establish the context. Bermuda measures interference between DMA streams on an Arm SMMUv2 platform, including transfer-size dependence [2]. Koenig et al. evaluate IOMMU-based shared virtual addressing on a RISC-V field-programmable gate array (FPGA) system, including memory-latency and host-traffic effects [5]. Rodríguez's thesis evaluates the same platform family and recognizes contention among its devices [6]. Zhao et al. evaluate multiple I/O address-translation caches sharing a translation server, while Hur and Ro evaluate concurrent clients and IOTLB policies using a RISC-V processor configuration and a system memory management unit (SMMU) model [7], [8]. These are direct performance precedents. The contribution of this paper is the depth and traceability of an implementation-specific characterization, rather than discovery of the shared-translation problem.

Aggregate slowdown alone cannot identify a translation bottleneck. A co-running neighbor competes for data movement even if both streams have no measured translation misses. Increasing its address working set changes translation activity, data addresses, and service timing together. Likewise, a longer internal request lifetime includes waiting and service, and overlapping request lifetimes cannot be added into an elapsed-time breakdown. We therefore combine a fixed-task co-run control, segmented hardware observations, and a bounded intervention on one identifiable response.

The main measurements use direct wiring of the IOMMU's data-structure request path (DS0). An additional configuration inserts diagnostic delay (DS50) to test sensitivity to that service path. In DS0, growing the aggressor from A1 to A8 and A64 increases median victim hardware elapsed time by 10.4% and 16.35%, respectively. The corresponding DS50 increases are 44.8% and 57.66%. These comparisons do not assign the difference to pure page-table-walk cost. The DS0 high-pressure conditions also retain small nonzero victim hardware performance monitoring (HPM) miss counts; their software measurement boundary differs from the hardware timing window.

The paper contributes three connected bodies of evidence. The fixed repaired FPGA baseline quantifies co-run loss under autonomous supply and records measurement windows and source identity. Historical request-level experiments identify a specific path by which an aggressor's delayed page-table-walk response extends a neighboring hit's wait, separately from capacity replacement. Historical mitigation experiments account for progress on both sides and preserve a combination regression whose full board-level cause remains unresolved; invalidation scope is a separate simulation supplement. All conclusions are bounded by the tested implementation, load, and evidence level.

II. Background

An IOMMU associates a device request with a device context, translates its I/O virtual address, and checks permissions before forwarding the request. The IOTLB caches translation information. On a miss, a page-table walker (PTW) traverses the applicable page tables; device-directory or other data-structure accesses can also be necessary. The RISC-V specification defines the architectural behavior and management interfaces, while the organization and parallelism of the implementation determine how requests share service [1].

Three forms of delay matter to this characterization. *Shared data-path competition* exists when concurrent DMA reads, writes, and responses use common downstream resources. *Service serialization* makes one translation wait for another request even when its eventual lookup hits. *Capacity replacement* removes a reusable translation, causing a later request to miss. These mechanisms can coexist, so a miss counter is neither a decomposition of total delay nor evidence that every recorded hit received immediate service.

The distinction has substantial precedent. HERO exposes translation events and demonstrates hit-under-miss behavior. Kurth et al. explicitly describe how full miss-related buffers can stall later accesses, including hits, and examine scalable handling and DMA support [4], [3]. Our local intervention tests the corresponding dependency in a different fixed controller, rather than claiming a new general principle.

We call the fixed foreground stream the *victim* (V) and its co-running stream the *aggressor* (A). These names identify experimental roles, not malicious intent. Solo runs V alone; A1 provides a co-running small-working-set control, while A8, A16, and A64 increase the aggressor's configured working set. Equal nominal transfer sizes do not imply equal translation activity or equal service schedules.

III. Platform and Methodology

A. Fixed repaired baseline

The platform instantiates a CVA6 host, four DMA engines, a shared RISC-V IOMMU with a 16-entry original replacement structure, and a double-data-rate (DDR) memory path on a Genesys2 FPGA. The evaluated clock is 50 MHz. The primary measurements activate a victim and one aggressor; the presence of four instantiated engines is not a four-active-device result.

The baseline uses the original IOTLB organization with necessary platform and correctness fixes, explicit autonomous batch supply, and passive path observation. It excludes the partitioning, bypass, composition, banked-service, and request-intervention extensions used in other historical configurations. We do not obtain an "original" baseline merely by disabling switches in a research tree. Each configuration has a fixed source manifest, build configuration, software payload, and result package; historical runs retain their separate identities and any missing identity fields.

Both new configurations share one software payload. DS0 directly connects the data-structure path. DS50 separately enables a 50/50-cycle diagnostic delayer on that path. The path serves more than page-table requests, and the resulting service schedule can change miss behavior. Thus DS0 is the original-wiring hardware result, while DS50 tests sensitivity to added data-structure-path latency. Neither the DS0–DS50 difference nor A1–A8 subtraction is a pure PTW-cost estimate.

B. Workload and controls

The baseline keeps the victim task fixed: 4096 copies of 4 KiB after a 256-item warm-up. The useful copied payload is 16 MiB. Autonomous hardware supply keeps the batch running without requiring the CPU to issue and poll each individual descriptor. This is an explicit workload fixture and changes the offered traffic relative to a software-fed loop. Copy throughput counts useful copied payload, not the sum of read and write memory traffic.

Solo, A1, A8, A16, and A64 preserve the victim's configuration while changing whether and how the neighbor runs. A1 is a conventional experimental control with zero measured misses on both sides in the new baseline. It demonstrates that a substantial co-run loss exists without measured misses; it does not make the remaining A1–A8 contrast a translation-only intervention. The latter also changes the address stream and its interaction with data service.

The completed baseline contains 100 accepted trials, with 10 trials per DS/condition. Collection followed DS50–DS0–DS50, with 25/50/25 trials over three actual boots. DS0's condition repeats share one boot; each DS50 condition pools five trials from each of the two DS50 boots, and the segments remain separately traceable. The repeats describe variation within these collected runs, rather than independent reboot sampling.

C. Timing and passive observers

Hardware elapsed time uses the victim's hardware observation window. Software elapsed time includes distinct control and polling effects and is reported as a separate metric. HPM miss counters are read through non-atomic software snapshots, which need not have exactly the hardware window's boundaries. A zero HPM count alone does not prove the first lookup of every hardware-window request hit. Conversely, small nonzero software-window counts cannot locate those misses inside the timed batch.

The path observer follows read and write address events through four stages: ingress presentation to acceptance, internal passage to translated-address presentation, translated-address backpressure to acceptance, and subsequent completion. The read completion boundary and write completion boundary follow their respective response protocols. Internal passage includes waiting, translation service, and departure-related effects; it is not a PTW timer. The final write stage also includes write-data supply and response effects.

For a stage in trial i, let S_i be the accumulated lifetime of complete observed transactions and N_i their count. The plotted mean is S_i/N_i, followed by the median across trials. Complete and cross-window transactions are distinguished by the observer rather than silently clipping a lifetime. Read and write activity and multiple in-flight transactions overlap. Stage means therefore cannot be stacked into fractions of total elapsed slowdown, and accumulated sums or maxima cannot reconstruct a latency cumulative distribution function (CDF).

D. Controlled historical evidence and statistics

Historical E3 experiments use a separate 64-byte diagnostic task and identify the aggressor request, selected PTW response, and waiting victim request. A one-response delay intervention is enabled or disabled within complementary

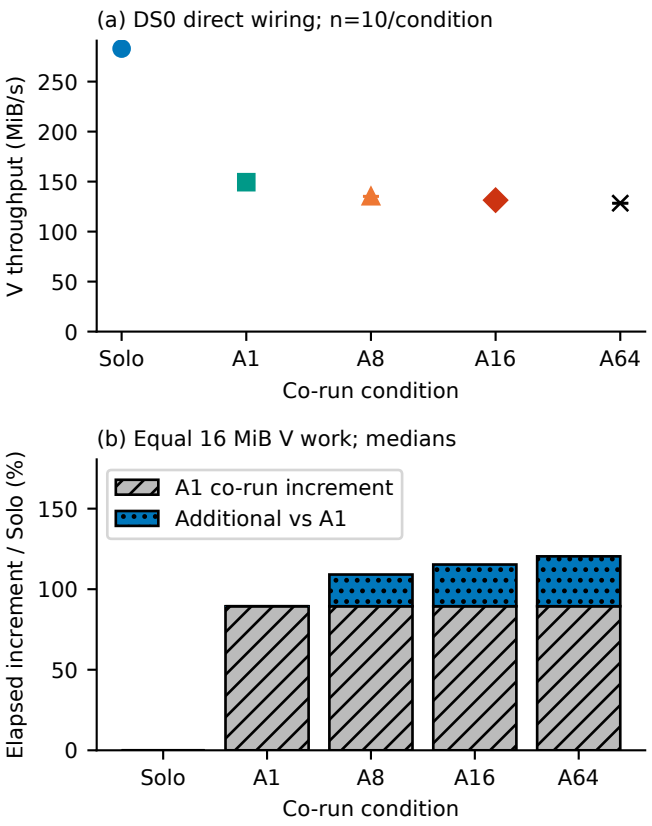


Fig. 1. Victim throughput and elapsed-time contrasts from the fixed repaired baseline. DS0 is the primary configuration. The A1 co-run term and the additional higher-working-set term are descriptive contrasts, not a measured partition of PTW cost.

pairs; the victim's first-lookup hit and its own translation-service interval are observed separately. These pairs test a local dependency. They are not equivalent to the new sustained-batch workload and do not apportion its complete slowdown.

We report descriptive medians and the recorded repeat count. Throughput is calculated for each trial before aggregation. Reported extra elapsed slowdown is

$$100\left(\frac{\text{median}(T_{A_k})}{\text{median}(T_{A1})}-1\right), \tag{1}$$

using matching DS and source configuration. It is a ratio of medians, not a median of paired ratios. Historical paired interventions retain their own pairing and sample counts. Where error bars are shown, they are percentile-bootstrap intervals for the reported median or contrast, resampled within the observed boots with a fixed seed. They describe these collected runs, not a population of independent reboots, an equivalence test, or a worst-case guarantee.

IV. Characterization Results

A. Co-run loss under original wiring

Figure 1 establishes the main result without inserted data-structure-path delay. Solo measures the fixed victim alone. A1 adds a co-running neighbor while both streams retain zero HPM miss counts. The Solo–A1 victim-throughput decrease therefore cannot be attributed to measured misses. The comparison is essential even though it is a conventional control rather than a methodological invention.

Increasing the neighbor's configured working set creates additional loss. Under DS0, A8 and A64 add 10.4% and 16.35% to the A1 median victim hardware elapsed time. The denominator is the matching A1 result, not Solo. Hardware throughput is based on the same fixed useful victim payload in each trial; a percentage increase in elapsed time and a percentage decrease in throughput have different denominators.

Victim slowdown must be distinguished from aggregate progress. Using completed aggressor write responses as a common-window payload proxy, A1's aggregate progress is 105.58% of the matched Solo throughput. A8 and A64 instead show aggregate proxy losses of 4.34% and 9.23%, respectively. The proxy counts useful COPY payload once and does not measure exact physical memory bandwidth. The underlying record does not capture exact same-window write-data bytes; its documented transaction-boundary error remains attached to the estimate. Thus a victim receiving roughly half of concurrent progress is different from a loss of half of system throughput.

These measurements support a bounded claim about this implementation and offered load: a neighbor with a larger translation working set can further delay a fixed victim beyond the zero-miss co-run loss. The contrast also changes the aggressor's address stream and service timing. It cannot isolate the incremental cycles spent walking page tables, the associated memory traffic, and internal serialization into separately additive costs.

The HPM boundary matters. The DS0 A8, A16, and A64 conditions retain victim miss medians of two, while A1 retains zero on both sides. These samples are not discarded. Because the counters use a software snapshot interval, the observations neither establish a zero-miss victim for those DS0 conditions nor identify where the small counts fall relative to the hardware timer. The direct request-level evidence below uses a separate observation of the victim's first lookup.

B. Where request lifetime grows

Figure 2 distinguishes the shared data-path effect from internal waiting. In the historical path experiment, Solo–A1 leaves the internal mean unchanged while increasing later request lifetime. A1–A8 instead increases internal passage markedly; the translated-address output does not acquire extra observed address-handshake backpressure. The new baseline reproduces the same qualitative separation, with a stronger internal increase under DS50.

The observer localizes a problem but does not itself establish its complete cause. Internal passage includes queueing before lookup and service, the translation operation, and exit-related waiting. The write completion stage also includes write-data availability and the return response. A delayed translated read can affect later write-data timing, while memory traffic can compete independently. We therefore retain the read and write

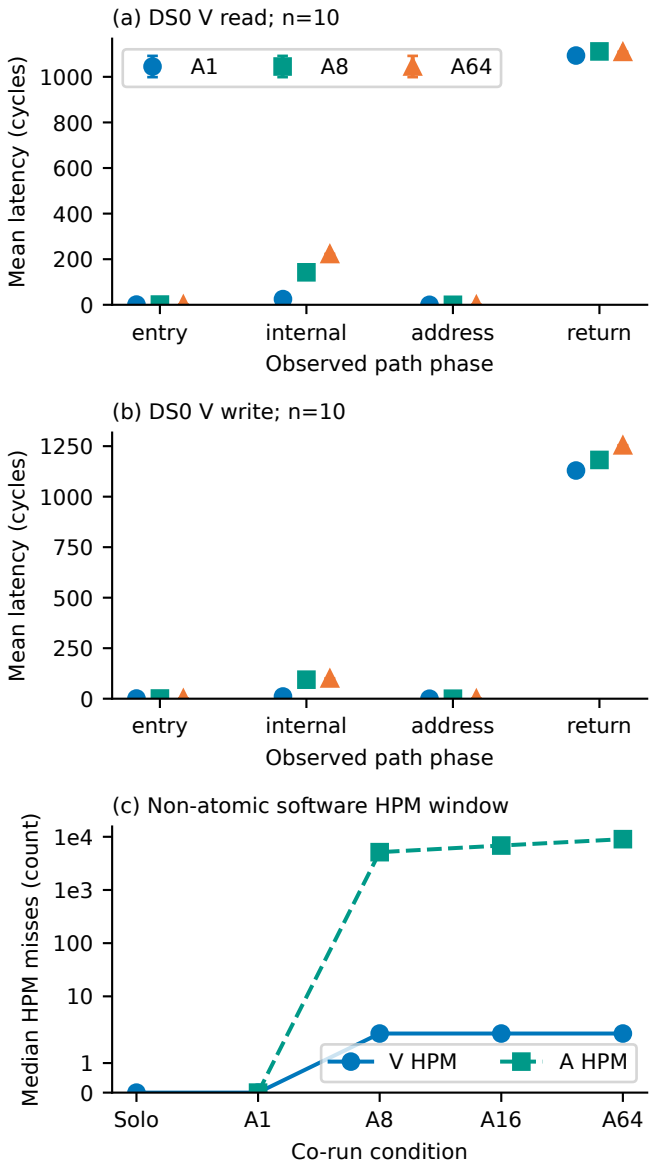


Fig. 2. Segmented victim read/write lifetimes and separately measured HPM misses. Each path value is a cross-trial median of a per-trial request mean. Stages and channels overlap; the panel is not an additive time breakdown.

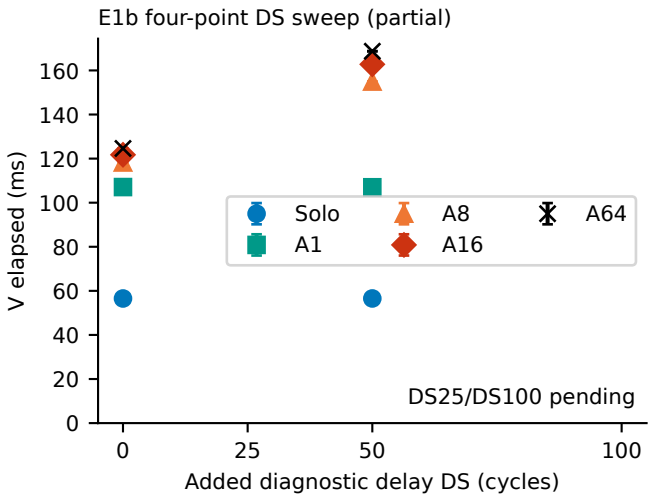


Fig. 3. Data-structure-path delay sensitivity. Completed DS0 and DS50 baseline points are distinguished from any pending DS25/DS100 measurements. A missing point is not interpolated into a measured curve.

panels and do not label every downstream increment as a DDR service increase.

The path mean is not a tail statistic. A larger mean may reflect many moderately delayed requests or a smaller set of long waits. The saved sums, counts, and finite maxima do not determine the distribution between them. This distinction constrains both the current conclusions and the pending latency-distribution experiment.

C. A named response extends a neighboring hit's wait

The historical E3 experiment addresses the local dependency left open by aggregate path counters. It selects one aggressor stage-one PTW response while a victim read is waiting, then compares delay-enabled and delay-disabled runs with recorded request identities and prefix events. The victim subsequently reports a first-lookup hit.

Across the four formal complementary cold pairs, the added aggressor remaining service and victim remaining wait match pair by pair, ranging from 127.0 to 130.0 cycles. The victim's own observed translation-service interval remains 4 cycles. This is evidence that the extended aggressor response delays the waiting victim before the victim's own hit service in the tested controller. Source inspection identifies the shared serial control state and read-address queue as the corresponding service dependency.

This causal claim remains local. Only one complementary pair has matching recorded pre-trigger timing; the other pairs are retained with their differences. The hot controls do not trigger the delay and are negative controls for triggering, rather than equal-exposure tests. The diagnostic uses a different transfer size and an historical image. Its cycle deltas do not explain the complete sustained-batch slowdown, and the experiment does not imply that the RISC-V architecture or Advanced eXtensible Interface (AXI) protocol requires this organization.

D. Sensitivity to data-structure-path delay

The additional delay configuration magnifies the higher-working-set contrast. A8 and A64 increase median hardware elapsed time over DS50's A1 by 44.8% and 57.66%, respectively, compared with the smaller DS0 increments. Figure 3 reports configurations separately, preserving their actual collection order and boots.

The sensitivity is informative because translation-related service depends on the latency of the memory system and control path around it. It is not a clean PTW-only intervention: data-structure requests include other accesses, and altered schedules can change both aggressor and victim miss observations. The experiment therefore supports sensitivity to

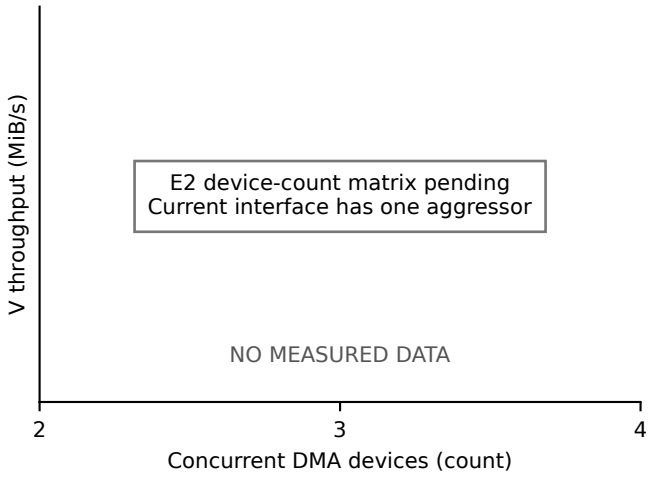


Fig. 4. Device-count scaling slot. Four DMA engines are instantiated, but no completed two-to-three-to-four active-device matrix is available for this draft.

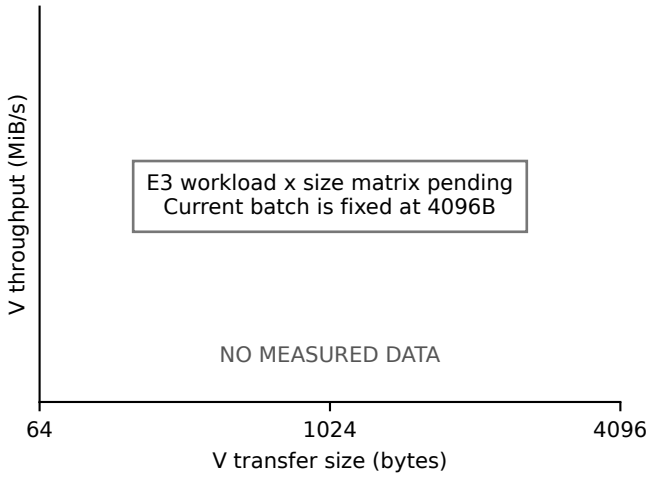


Fig. 5. Workload-shape and transfer-size slot. Sequential, random, and periodic-burst comparisons require a matched baseline; no illustrative values substitute for measurements.

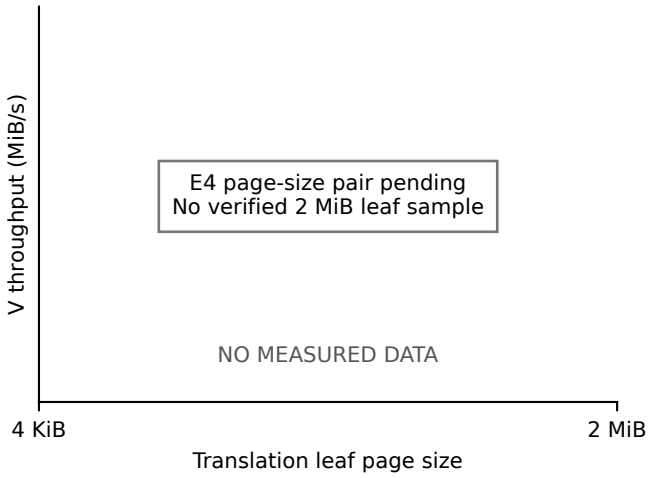


Fig. 6. Page-size comparison slot. A valid comparison must verify the actual page-table leaf level and alignment; buffer allocation or address stride alone does not establish a larger translation page.

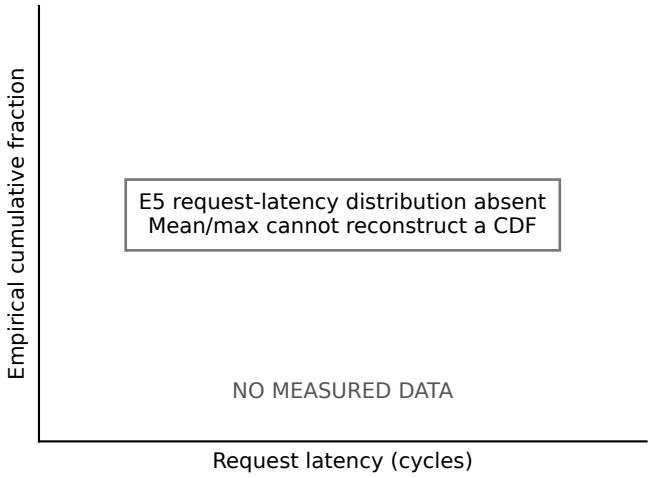


Fig. 7. Request-latency distribution slot. Existing means and limited observed maxima cannot reconstruct a CDF or p99.

the configured path delay; it does not estimate a universal slowdown per PTW cycle or demonstrate a page-table-priority policy.

E. Coverage still required for generalization

Figures 4–7 preserve the planned tests whose results are not present. The active-device count, the aggressor's individual and total offered pressure, and the completion of every stream must remain distinguishable. Workload-shape and transfer-size studies must retain matched work and expose the supply pattern. Page-size studies must verify the installed translation leaf, not infer it from a DMA buffer. Request-latency percentiles require bounded distribution capture with overflow or loss accounting.

These gaps limit comparison with other platforms. Bermuda includes transfer-size-dependent results, so a batch of larger copies on our platform cannot be treated as a direct contradiction of its curve [2]. The unresolved question is which service and supply conditions preserve or reduce interference on this implementation. The working draft makes no device-scaling, page-size, or latency-tail finding before those measurements exist.

F. Historical mitigation costs

The historical rate-control experiment improves victim progress by reducing the aggressor's issue pressure, with a corresponding cost to aggressor progress in the common window. Aggressor work completed after the victim window is not divided by that window's duration and relabeled concurrent throughput. This accounting prevents protection of one stream from appearing free because the other's tail is ignored.

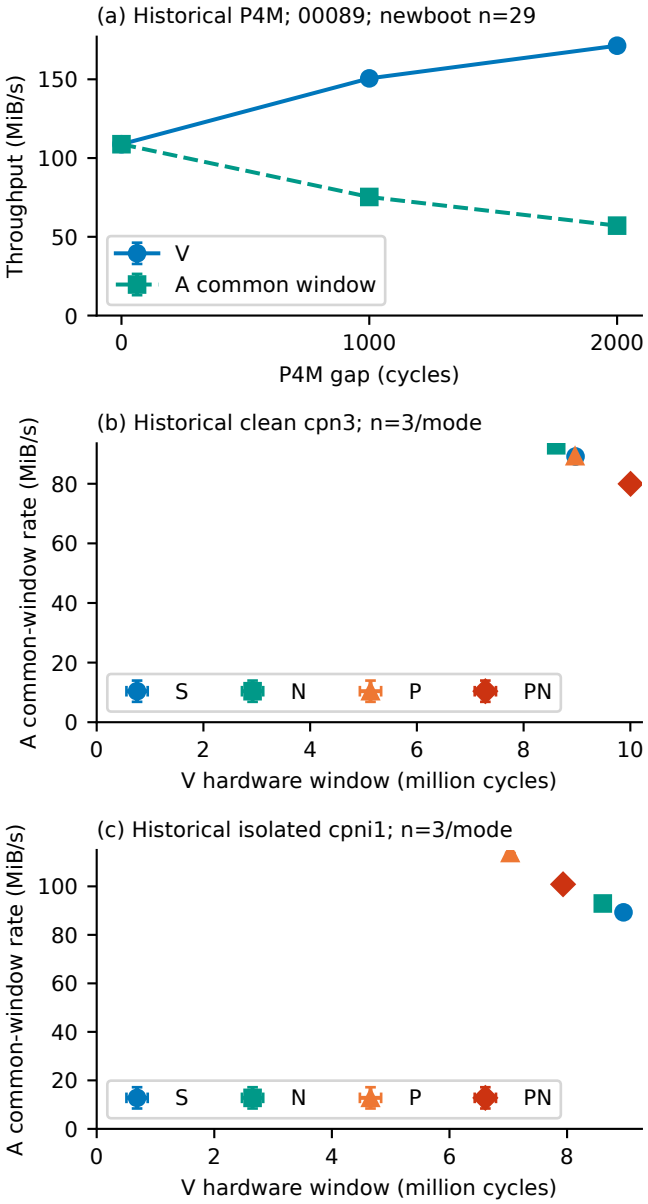


Fig. 8. Historical mitigation evidence. Victim protection and aggressor progress retain their own common-window denominators. Separate version and collection groups are not pooled into one new-baseline matrix.

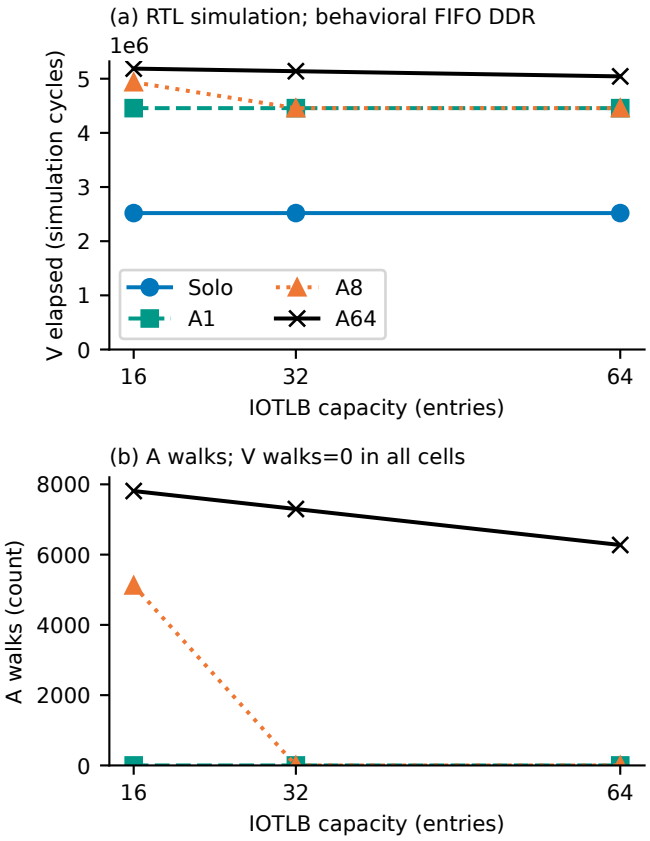


Fig. 9. IOTLB-capacity sensitivity in a separate real register-transfer-level (RTL) simulation. Its software-register-fed workload and behavioral memory service differ from autonomous FPGA batch supply. No repeated-sample confidence interval or FPGA time conversion is implied.

A distinct composition experiment compares partitioning (P), bypass-related service (N), and their combination (PN). After isolating replacement history, PN remains 12.67%–12.79% slower than P across three descriptive paired comparisons even though their aggressor miss counts match. Equal miss counts do not establish equal walk addresses, memory-service schedules, or request timing. The full cause of this board-level difference remains unresolved. A reachable contention path in a behavioral memory model is not proof that it explains the measured board difference.

These are configuration-specific tradeoffs and a preserved negative result. They do not establish that partitioning or rate control is a new technique, that composing optimizations is generally harmful, or that the historical results already form a matched mitigation matrix on the repaired baseline. The latter remains a separate measurement gap.

G. Capacity replacement is a separate effect

The historical E2 experiments observe residency before a controlled access sequence, removal under one boundary layout, and a miss on the first later reuse of the victim translation. This supports a capacity-replacement chain under those layouts. It differs from E3, where a victim request that first hits waits for an aggressor's service. Cache occupancy and service serialization must therefore be considered separately even when both accompany a larger neighbor working set.

The tested W7/W8 layouts do not define a universal device capacity threshold. Source and destination translation keys, access order, replacement state, and direction are part of the experimental condition. We retain this finite-layout evidence rather than use it to partition the new baseline's total slowdown.

H. Capacity sensitivity: separate register-transfer-level simulation

Figure 9 reports the completed capacity sweep from the separate real-RTL simulation package. The simulation uses ordinary memory-mapped I/O (MMIO) register issue and a single COPY task in flight rather than the board's autonomous batch fixture. The victim warm-up uses 64 copies plus two zero-walk checks, rather than the board's 256-copy warm-up; no CPU or Linux executes in this model. The memory model serves reads in arrival order and waits 16 cycles at a read-queue head, then returns one 8-byte beat per cycle. Its baseline-capacity calibration reproduces the ordering of Solo, A1, and A8, while retaining different absolute timing and memory service from the board.

For A8, increasing capacity eliminates the aggressor walks at the intermediate and largest capacities, and victim elapsed time becomes equal to the matching A1 time. The A1–Solo difference remains. The larger A64 working set retains aggressor walks and additional delay at the largest evaluated capacity. These results distinguish a capacity-sensitive increment from remaining co-run delay in this model; they do not separate that residual into serial-controller waiting and memory competition. The simulation cannot replace the historical board residency experiment or establish a capacity benefit for unmeasured FPGA configurations.

I. Invalidation scope: simulation supplement

The separate invalidation study runs real RTL DMA and command-queue logic against a behavioral DDR service model. It varies invalidation scope and domain while accounting for the issuing stream's complete work, final flush, and the neighbor's complete work separately. A narrow invalidation can eliminate the misses attributed to removed neighboring resident translations, yet increase the issuing stream's complete time relative to a wider invalidation.

Attribution uses the recorded resident-key invalidation and first subsequent miss, rather than counting every extra walk over a baseline as invalidation-induced. The supplement tests an explicit scope/command tradeoff in the model. It does not supply a current-mainline Linux cost, a hardware time conversion, or an FPGA validation of the simulated conclusions.

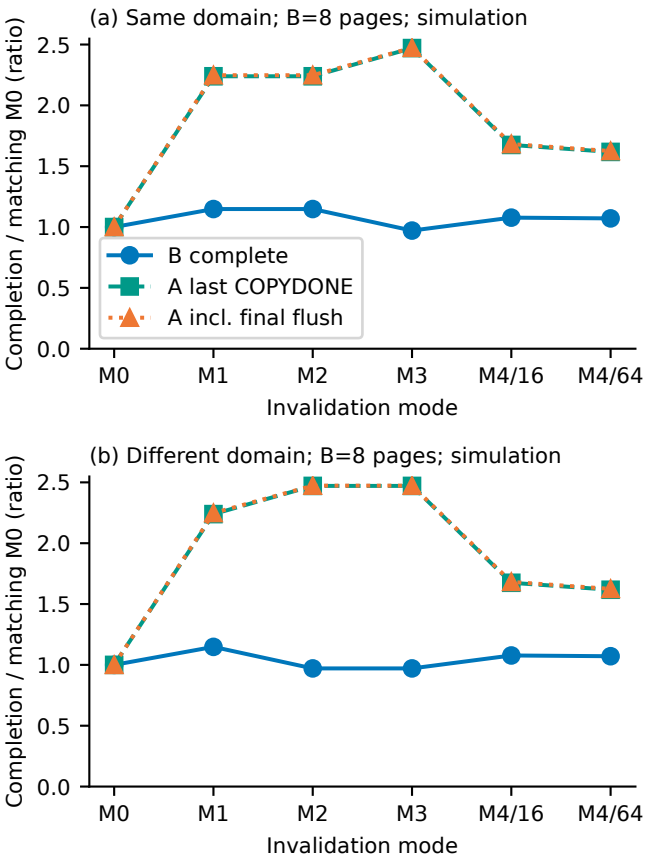


Fig. 10. Invalidation-scope results from Verilator with behavioral DDR service. Time axes are simulation steps. These are neither Linux execution nor FPGA performance results.

V. Discussion and Threats to Validity

What the evidence adds. The completed measurements progress from a macro-scale contrast to a location and then a local dependency: a zero-miss co-run control bounds claims about misses, segmented observations expose internal waiting, and a named-response intervention links an aggressor's remaining PTW service to a neighboring hit's remaining wait. None of these stages licenses a complete additive attribution of the batch slowdown. The primary conclusion concerns the tested controller's service organization, with DS50 exposing sensitivity beyond original wiring.

Internal validity. Miss counters use non-atomic software windows, while path counters and elapsed timers have hardware boundaries. Conflating them can create a false zero-miss or per-request claim. Passive instrumentation can affect area and routing, and similar throughput in different historical images is not a matched measurement of observer overhead. Necessary fixes and the autonomous supply fixture are explicit parts of the baseline. Cycle-to-time conversion uses the configured clock, while the original records leave independent HPM calibration unverified. Positive reported timing slack applies to the constrained paths and does not establish a complete clock-domain-crossing or external-I/O signoff.

Construct and statistical validity. COPY is a controlled traffic generator, not an application, a PCIe endpoint, or an accelerator workload trace. The active workload changes more than a miss count. Most repeats share a boot, and the pooled DS50 repeats span two boots in a fixed collection order. Finite medians, four local complementary interventions, limited replacement layouts, and three composition pairs support descriptive findings, not broad probabilistic or worst-case guarantees. The incomplete distribution and workload matrices remain visible instead of being extrapolated from means.

External validity and provenance. A single FPGA integration and controller do not represent every RISC-V IOMMU implementation or a commercial memory system. Historical images differ from the repaired baseline and have incomplete

loaded-image and software identity checks; those uncertainties remain attached to their evidence. Simulation uses its documented memory model and excludes CPU/Linux execution. Published predecessors use different implementations, traffic, baselines, and normalization. We compare their questions and evidence directly, but do not divide their reported absolute bandwidth by ours.

VI. Related Work

Measured shared-translation performance. Bermuda demonstrates cross-DMA interference on Arm SMMUv2 and evaluates transfer-size dependence with translation enabled and disabled [2]. Koenig et al. study RISC-V shared virtual addressing on FPGA, including host noise, PTW time, and caching of the page-table path [5]. Rodríguez's thesis supplies the closest platform predecessor: multiple iDMA engines, IOMMU performance evaluation, cache-capacity effects, and recognition of device contention [6]. Our comparison concerns sustained autonomous traffic and the internally observed service dependency, not a claim that these works lacked performance evaluation.

Direct concurrent-client precedents. Zhao et al. implement distributed I/O address-translation caches (IOATCs) sharing an I/O address-translation server (IOATS), evaluate multi-IOATC scaling, and provide internal hit/miss waveforms and FPGA network validation [7]. Hur and Ro run concurrent client mixtures, evaluate shared-IOTLB replacement policies, and study an adaptive policy using a RISC-V processor configuration with an SMMU model [8]. These directly overlap the shared-translation performance question. Relative to their reported evaluations, our focus is a fixed-neighbor control and a request-identified response intervention connecting another stream's PTW service to a hit's waiting time. Their simulation or architectural differences do not remove them from the comparison.

Mechanism and observation history. HERO already provides internal translation-event visibility and hit-under-miss behavior; Kurth et al. discuss miss-related buffering that can block subsequent hits, as well as prefetching and DMA support [4], [3]. We therefore claim implementation-specific evidence and quantified limits. Neither hit-under-miss, shared-service blocking, partitioning, rate control, nor the non-additivity of optimizations is presented as a newly discovered general principle.

VII. Conclusion

The FPGA evidence shows why total throughput and translation-miss counts must be interpreted together with service organization. A zero-miss co-run control exposes ordinary shared-path loss; internal observation and a bounded request intervention identify one controller-specific path from a neighbor's PTW service to a hit's waiting time. Original wiring is the primary measurement condition, added delay is a sensitivity diagnostic, historical mitigation comparisons preserve costs on both sides, and invalidation scope remains a simulation supplement. The remaining workload, page-size, scaling, and latency-distribution gaps bound the present characterization.

Artifact Availability and Declarations

Source, analysis scripts, raw measurement records, and reproduction instructions will be released upon acceptance. This anonymous draft intentionally omits identifying repository links. The study uses synthetic machine-generated workloads and does not involve human participants. Artificial intelligence (AI) tools assisted evidence organization, manuscript drafting, and code-related work; the final authors remain responsible for verification.

Draft gap. Author contributions, funding, and conflict-of-interest declarations require author-supplied facts before submission; they are not inferred from this draft. The abstract and result text also require a final update after pending experiments.

References

[1] RISC-V International, *RISC-V IOMMU Architecture Specification*, v1.0.1, revision 20260222 ed., 2026, architectural background; not a claim that the evaluated RTL implements every extension. [Online]. Available: https://docs.riscv.org/reference/iommu/v20260222/

[2] D. Costa, J. Martins, and S. Pinto, "Beyond the Bermuda triangle of contention: IOMMU interference in mixed criticality systems," in *IEEE International Conference on Embedded and Real-Time Computing Systems and Applications (RTCSA)*, 2025, technical claims checked against author manuscript arXiv:2508.19670v1. DOI: https://doi.org/10.1109/RTCSA66114.2025.00027. [Online]. Available: https://arxiv.org/abs/2508.19670v1

[3] A. Kurth, P. Vogel, A. Marongiu, and L. Benini, "Scalable and efficient virtual memory sharing in heterogeneous SoCs with TLB prefetching and MMU-aware DMA engine," in *IEEE International Conference on Computer Design (ICCD)*, 2018, pp. 292–300, technical claims checked against author manuscript arXiv:1808.09751v1. DOI: https://doi.org/10.1109/ICCD.2018.00052. [Online]. Available: https://arxiv.org/abs/1808.09751v1

[4] A. Kurth, P. Vogel, A. Capotondi, A. Marongiu, and L. Benini, "HERO: Heterogeneous embedded research platform for exploring RISC-V manycore accelerators on FPGA," in *Computer Architecture Research with RISC-V Workshop (CARRV)*, 2017. [Online]. Available: https://carrv.github.io/2017/papers/kurth-hero-carrv2017.pdf

[5] C. Koenig, E. Zelioli, and L. Benini, "Evaluating IOMMU-based shared virtual addressing for RISC-V embedded heterogeneous SoCs," in *Design, Automation & Test in Europe Conference (DATE)*, 2025, technical claims checked against author manuscript arXiv:2502.17398v1. DOI: https://doi.org/10.23919/DATE64628.2025.10992962. [Online]. Available: https://arxiv.org/abs/2502.17398v1

[6] M. A. C. Rodríguez, "An open-source RISC-V input/output memory management unit," Master's thesis, Universidade do Minho, Jan. 2024. [Online]. Available: https://hdl.handle.net/1822/99350

[7] S. Zhao, M. Hong, X. Wang, D. Wang, and Y. Liu, "Performance optimization and research of data center-level IOMMU based on RISC-V," *Procedia Computer Science*, vol. 266, pp. 923–935, 2025, DOI: https://doi.org/10.1016/j.procs.2025.08.115. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1877050925024263

[8] W. Hur and W. W. Ro, "Enhancing IOMMU efficiency in heterogeneous SoCs: A study on cache policy impacts," in *International Conference on Electronics, Information, and Communication (ICEIC)*, 2025, DOI: https://doi.org/10.1109/ICEIC64972.2025.10879735. [Online]. Available: https://ieeexplore.ieee.org/document/10879735